Système d'exploitation : TP4, introduction aux sémaphores

février 2026 — Pierre Rousselin

Exercice 1 : `million.c` : section critique

1. Télécharger sur moodle le fichier `million.c` (vu en cours). Le lire, le compiler et exécuter le programme plusieurs fois pour vérifier que le résultat n'est pas déterminé.
2. On a vu en cours que le problème vient du fait que l'instruction `++incremente_moi;` n'est pas atomique. Il faudrait donc s'assurer qu'aucun *thread* ne commence cette instruction si un autre ne l'a pas terminée. On parle de *section critique*. En utilisant un sémaphore binaire (c'est-à-dire dont le compteur vaut toujours 0 ou 1), faire en sorte qu'il ne puisse jamais y avoir plus d'un *thread* dans cette section critique. Bien sûr, vérifier le résultat en exécutant plusieurs fois le programme.

   Pour utiliser le sémaphore, plusieurs stratégies sont possibles (vous pouvez en essayer plusieurs), par exemple
   - le sémaphore est global, donc visible dans tout le programme, initialisé (et détruit) dans `main` ;
   - le sémaphore est local à `main`, son adresse est passée en argument lors du `pthread_create`.

--- * ---

Exercice 2 : Le problème du rendez-vous (à 2)

Le problème du rendez-vous apparaît lorsqu'on veut que deux tâches (ou plus) se synchronisent à un certain point de leur exécution (on parle aussi de barrière).

Pour montrer cette situation, on considère le programme `rendez_vous_a_2_non-sync.c` dans lequel deux *threads* se séparent la lourde tâche de compter de 0 à 100 en même temps sans s'écouter.

1. Télécharger ce programme sur moodle, le lire, le compiler et l'exécuter. Les nombres sont-ils dans l'ordre ?
2. On voudrait qu'il n'y ait pas de décalage et que les a et b donnent les nombres par paires (pas forcément dans le même ordre entre a et b), par exemple autoriser

   ```
   a : 0
   b : 0
   b : 1
   a : 1
   ```

   mais pas

   ```
   a : 0
   a : 1
   b : 0
   b : 1
   ```

   À l'aide de 2 sémaphores, obtenir ce résultat. Pour ce faire, on initialisera, dans la pile du *thread* initial, un tableau de 2 sémaphores dont l'adresse sera transmise aux deux autres *threads*.

--- * ---

Exercice 3 : Précédence, d'après le partiel 1 2021 – 2022

Un travail comporte 5 tâches : les tâches A, B, C, D et E. Les contraintes sont les suivantes :
- la tâche E ne peut commencer que si C et D sont terminées ;
- la tâche C ne peut commencer que si A et B sont terminées ;
- la tâche D ne peut commencer que si B est terminée.

1. Dessiner le diagramme de dépendance (ou encore de précédence) associé à cette situation.
2. Le programme `dependance.c`, à télécharger sur moodle, exécute ces tâches séquentiellement. En utilisant des *threads* et des sémaphores, séparer les différentes tâches de manière à ce que le programme prenne le moins de temps possible, tout en respectant les contraintes.

--- * ---